

Como o Linux Gerencia Memória na Prática

Zonas, alocadores, VMAs, page cache, parâmetros VM e ciclo de vida do processo

1. ZONAS DE MEMÓRIA FÍSICA

ZONE_DMA 0 – 16 MB legacy DMA (24 bits)	ZONE_DMA32 16 MB – 4 GB DMA 32 bits	ZONE_NORMAL acima de 4 GB uso geral	ZONE_MOVABLE configurável huge pages, hotplug
--	--	--	--

Em máquinas NUMA, cada nó tem seu conjunto de zonas. Kernel tenta alocar páginas no nó local. Observar: `/proc/zoneinfo` e `numactl --hardware`

3. VMAs — Virtual Memory Areas

Cada processo tem uma lista de **vm_area_struct** — uma para cada faixa contígua com mesmas permissões e mesmo backing.

Quando ocorre page fault, kernel localiza a VMA que contém o endereço e despacha o handler.

Cada linha de `/proc/maps` corresponde a uma VMA. Campos: vm_start, vm_end, vm_flags (R/W/X/SHARED), vm_file, vm_pgoff, vm_ops.

5. /proc/sys/vm/ — Parâmetros do Subsistema VM

Parâmetro	Função	Default
swappiness	tendência de mandar páginas anônimas para swap (0–100)	60
overcommit_memory	0=heurística, 1=sempre permitir, 2=estricto	0
dirty_ratio	% de RAM até bloquear escritas (writeback síncrono)	20
min_free_kbytes	memória mínima livre que kswapd mantém	calc.
panic_on_oom	pânico do kernel ou só mata processo em OOM	0

7. OOM KILLER

Quando RAM e swap esgotam, kernel não consegue mais entregar páginas. Em vez de travar, dispara o **OOM Killer**:

- Cada processo tem um score em `/proc/oom_score` (baseado em uso, idade, prioridade)
- Score ajustável por `/proc/oom_score_adj` (-1000 = imune)
- Processo de maior score recebe **SIGKILL**
- Logado em **dmesg**: Out of memory: Killed process

2. ALOCADORES

Para o kernel

Buddy allocator — base de tudo, aloca em potências de 2 de páginas contíguas

Slab / SLUB — objetos pequenos do kernel (inode, task_struct)

kmalloc — fisicamente contíguo, pequeno

vmalloc — virtualmente contíguo, fisicamente espalhado

Para processos (userspace)

brk / sbrk — move topo do heap (legacy)

mmap — cria nova VMA (arquivo ou anônima)

munmap — remove mapeamento

madvise — dicas ao kernel (sequencial, descarte)

4. PAGE CACHE — o coração do Linux



6. CICLO DE VIDA DO PROCESSO

fork()

- Nova task_struct
- Tabelas duplicadas, apontando aos **mesmos frames físicos**
- Todas as páginas marcadas R-O
- COW: na 1ª escrita, kernel duplica AQUELA página

exec("/path/programa")

- VMAs construídas do ELF
- VMAs antigas liberadas
- Espaço virtual zerado
- NENHUMA página carregada (demand paging)
- 1º acesso ao .text → page fault, kernel lê do ELF

exit()

- Quando o processo termina
- Kernel percorre as VMAs e desfaz cada mapeamento
 - Páginas **anônimas** (heap, stack) → buddy allocator
 - Páginas de arquivo (libs .so, .text) → page cache (outros processos podem usar)
 - Tabelas de páginas liberadas | task_struct liberada via slab